Transfer Learning for Text Diffusion Models

Kehang Han[1*], **Kathleen Kenealy**[1*], **Aditya Barua**[2*], **Noah Fiedel**[1], **Noah Constant**[1]
[1]Google DeepMind [2]Google
{kehanghan, kkenealy, adityabarua, nfiedel, nconstant}@google.com

arXiv:2401.17181v1 [cs.CL] 30 Jan 2024

Abstract

In this report, we explore the potential for *text diffusion* to replace autoregressive (AR) decoding for the training and deployment of large language models (LLMs). We are particularly interested to see whether pretrained AR models can be transformed into text diffusion models through a lightweight adaptation procedure we call "AR2Diff". We begin by establishing a strong baseline setup for training text diffusion models. Comparing across multiple architectures and pretraining objectives, we find that training a decoder-only model with a prefix LM objective is best or near-best across several tasks. Building on this finding, we test various transfer learning setups for text diffusion models. On machine translation, we find that text diffusion underperforms the standard AR approach. However, on code synthesis and extractive QA, we find diffusion models trained from scratch outperform AR models in many cases. We also observe quality gains from AR2Diff—adapting AR models to use diffusion decoding. These results are promising given that text diffusion is relatively underexplored and can be significantly faster than AR decoding for long text generation.

1 Introduction

In recent years, large language models (LLMs) have grown in scale, capability, and popularity (Brown et al., 2020; Chowdhery et al., 2022), and are increasingly used to generate long-form text such as summaries, blocks of code, or in-depth explanations (OpenAI, 2023; Anil et al., 2023). To our knowledge, all popular LLMs are *autoregressive* (AR)—generating one token at a time in textual order, each conditioned on the sequence generated so far. While AR generation is well understood and has been highly optimized, its strict left-to-right factorization may be overly constraining. Generating token-by-token is inherently inefficient, particularly on long but predictable spans of text (e.g., copying a serial number from the context one digit at a time). Additionally, this strict order may not provide the ideal scaffold for planning a composition. Human writers typically outline, draft, revise, and proofread their work, and it seems plausible that machines could benefit from a similar iterative approach.[1]

As an alternative, many *non-AR* decoding methods have been proposed (see section §2), which generate multiple sequence positions in parallel, or make progressive edits to a "rough" initial generation. Several of these have shown promising results on specific tasks. For example, SUNDAE's *text diffusion* approach (Savinov et al., 2022) achieves similar quality to an AR baseline on machine translation while decoding over 2× faster.

However, despite positive findings, non-AR techniques have failed to gain traction, and remain unused in the space of large language models. We suspect this may be due to the inertia behind classic AR methods, and the high cost and risk of tuning and training large models from scratch using non-standard training losses and decoding methods.

With an eye to lowering this cost of entry and easing the transition to more efficient text generation at scale, in this paper we investigate the potential for adapting existing pretrained AR model checkpoints to perform non-AR generation. We use a simplified version of SUNDAE text diffusion as our canonical non-AR implementation; thus we refer to this lightweight adaptation process as **AR2Diff (AR to Diffusion)**.

More specifically, we are interested in testing the ability of text diffusion methods to compete at scale in the popular transfer learning setting, where a model is pretrained on unsupervised data and

[*]Equal contribution.

[1]"Chain of thought" prompting (Wei et al., 2022) provides a mechanism for models to reason about or draft the desired output before producing it. However, the final output is still generated autoregressively.

applied to diverse downstream tasks. We conduct a series of experiments comparing text diffusion to AR baselines across different model architectures, tasks, and transfer learning settings.

Our main contributions are: (1) showing that language models pretrained and fine-tuned using text diffusion can be competitive with autoregressive models on several downstream tasks, (2) showing that pretrained AR models can be transformed into diffusion models via a lightweight adaptation.

2 Related Work

Previous work has explored a wide range of non-autoregressive methods for text generation (Gu et al., 2018; Lee et al., 2018; Stern et al., 2019; Ghazvininejad et al., 2019). In the last few years, diffusion models (Sohl-Dickstein et al., 2015) have emerged as the primary technique for *image* generation (Rombach et al., 2021; Ramesh et al., 2022; Saharia et al., 2022). Many recent efforts have applied diffusion methods to *text* generation (Savinov et al., 2022; Li et al., 2022; Reid et al., 2023; Chen et al., 2023; Strudel et al., 2022; Dieleman et al., 2022; Zheng et al., 2023; Lin et al., 2023; Gong et al., 2023; Yuan et al., 2023; Wu et al., 2023), but none has yet gained adoption in the space of large language models.

While promising, text diffusion techniques have largely not been tested at scale or in multitask transfer learning settings, though see Lin et al. (2023) and Ye et al. (2023) for recent work in this direction. Furthermore, it remains unclear if these methods demand training new diffusion models from scratch, or if AR models can be efficiently adapted into diffusion models. We explore these questions empirically in section §4.

One line of previous work shows that non-AR methods benefit from "AR distillation" (Kim and Rush, 2016; Gu et al., 2018; Saharia et al., 2020; Gu and Kong, 2021)—training a non-AR model from scratch on silver data generated via the predictions of an existing AR model. AR distillation is similar to our AR2Diff adaptation in that both leverage a preexisting AR model. However they differ in that our method initializes the diffusion model directly from an AR checkpoint, and trains on gold data. Given the significant recent investment in training large AR models, we believe that lightweight adaptation of existing checkpoints is a promising direction compared to training non-standard models from scratch.

Recently, Lin et al. (2023) show good results pretraining a text diffusion encoder-decoder model and fine-tuning it on downstream tasks. Like our work, this validates the effectiveness of pretraining text diffusion models at scale.

More recently, building on "reparameterized discrete diffusion models" (Zheng et al., 2023), Ye et al. (2023) show the possibility of converting large AR models (up to 10B parameters) into text diffusion models during task-specific fine-tuning—their "diffusive adaptation". This work shares our goal of demonstrating that text diffusion can be practical at scale. Our work differs in (i) building on SUNDAE as opposed to RDM, (ii) including diffusion models pretrained from scratch as baselines, (iii) comparing different architectures and objectives for diffusion pretraining, and (iv) testing adaptation during pretraining (our AR2Diff$_N$ with $N > 0$), as opposed to only during fine-tuning (our AR2Diff$_0$).

3 Evaluation Tasks

We experiment with three downstream tasks. First, we use **WMT14 French-English translation** (Bojar et al., 2014), as machine translation is widely used to evaluate generative models, particularly in work on non-AR models.

Second, we evaluate on the popular **SQuAD question answering task** (Rajpurkar et al., 2016). As an extractive QA task, this does not require open generation, and most targets are fairly short, often just a few words long. While text diffusion models are unlikely to deliver speed gains on tasks with short outputs (see Section §4.7), we feel it is still important to test for quality on text *understanding* tasks. This can help establish whether pretrained diffusion models can be an effective general foundation for language understanding, and ensures that our findings are interpretable within the literature on transfer learning in NLP.

Finally, we evaluate on **Mostly Basic Python Problems (MBPP)** (Austin et al., 2021), a recent benchmark requiring models to generate full solutions to simple Python programming tasks. This task is fairly open-ended, as there are many working solutions to a given task, depending on choices of algorithm, coding style, variable names, and so on. Compared to open-ended natural language generation, this benchmark has clear and meaningful automatic evaluation metrics, as we can run the generated code and assess whether it passes rele-

vant test cases. When tokenized using the PaLM (Chowdhery et al., 2022) vocabulary we adopt in our experiments, median target length is 59 tokens, and 90th percentile is 150 tokens.

4 Experiments

4.1 Diffusion implementation

Our diffusion implementation follows SUNDAE (Savinov et al., 2022). More specifically, we use standard Transformer (Vaswani et al., 2017) architectures (either encoder-decoder or decoder-only) as implemented in the T5X (Roberts et al., 2022) library. As SUNDAE performs discrete diffusion in surface token space, the decoder inputs and outputs are tokens, in line with standard AR models. These implementation choices allow us to reuse existing frameworks for autoregressive LLM training with relatively minor changes. As a result, we can easily experiment with using pretrained AR model checkpoints and adapting these to perform text diffusion.

For training, we use the SUNDAE $L^{(1:2)}$ loss, which incorporates one step of "unrolled denoising", encouraging the model to be able to refine its single-step predictions further towards the target. More concretely, for target sequence x, we randomly corrupt a random proportion of tokens (sampling from a uniform distribution) to produce x^c, which is passed as input to the denoising model to produce logits l_1. The "logits loss" $L^{(1)}$ is the cross-entropy between l_1 and x. "Unrolled logits" are computed by sampling[2] from l_1 and passing these tokens back as inputs to the denoising model, producing l_2. The "unrolled logits loss" $L^{(2)}$ is the cross-entropy between l_2 and x. For the overall loss, we use $L^{(1)} + L^{(2)}$.

For inference, we follow SUNDAE in using low-temperature sampling ($\tau = 0.2$), decoding N samples in parallel (we use $N = 8$ by default), and reranking them based on "model score": the cross-entropy between the decoder input and output logits on the final step of diffusion. We use 10 diffusion decoding steps by default; thus on tasks with targets longer than 10 tokens, our diffusion models use fewer decoding steps than an AR model.[3] These choices are ablated in section §4.6.

For simplicity, we forgo SUNDAE's target length prediction module, opting instead to let the model learn to predict sequence length end-to-end through the presence of padding tokens observed during training. As a result, our text diffusion models have no additional parameters beyond those within the Transformer (encoder-)decoder.

4.2 Selecting objective and architecture

Previous work on text diffusion has focused on the single-task setting, either training and evaluating on unconditional text generation, or training from scratch on an end task, such as machine translation.[4] In contrast, we aim to evaluate text diffusion in the *transfer learning* setting—pretraining a large model, and adapting it to a range of downstream tasks. As a first step, and to cut down the space of further experiments, we first seek to identify a model architecture and pretraining objective well-suited to text diffusion.

The T5 study on transfer learning for AR text-to-text models (Raffel et al., 2020) recommends using an encoder-decoder architecture and a "span corruption" objective—masking multi-token spans in the input, and reconstructing these in the target. By comparison, many subsequent LLMs have converged on a decoder-only architecture with a standard LM objective (Brown et al., 2020; Chowdhery et al., 2022). To establish which setting works best for diffusion, we test all four combinations of architecture (**encoder-decoder** vs. **decoder-only**) and objective (**span corruption** vs. **prefix LM**), as shown in Figure 1.[5]

We train each model on the same pretraining mixture, consisting of 80% multilingual web crawl data from mC4 (Xue et al., 2021) and 20% Python code from "The Stack" (Kocetkov et al., 2022). All models use the T5 Base size transformer architecture and pretrain for 1 million steps on batches of size 128 and sequence length 1024. We then

[2] We sample from l_1 using temperature 0.0 (argmax), as opposed to SUNDAE's temperature 1.0, as we found this performed best in early ablations on WMT14, with temperature in { 0.0, 0.1, 1.0 }.

[3] As AR models can cache and reuse activations from earlier sequence positions for subsequent decoding steps (thanks to the causal attention mask), they use significantly fewer FLOPs per step, when other factors are held constant. We do not present a full picture of the speed vs. quality tradeoffs of text diffusion models here. Previous work has shown that text diffusion can be competitive on speed and quality, even comparing against AR inference with caching enabled (Savinov et al., 2022). We assume here that diffusion in 10 steps is fast enough to have practical value, and focus on quality.

[4] Ye et al. (2023) adapt pretrained AR models for diffusion across multiple tasks, but do not explore pretraining a general-purpose diffusion model that can be adapted to specific tasks.

[5] We choose the "prefix LM" objective rather than the standard causal LM objective, as it is compatible with the encoder-decoder architecture, and has been shown to outperform causal LM in apples-to-apples comparisons (Tay et al., 2023).

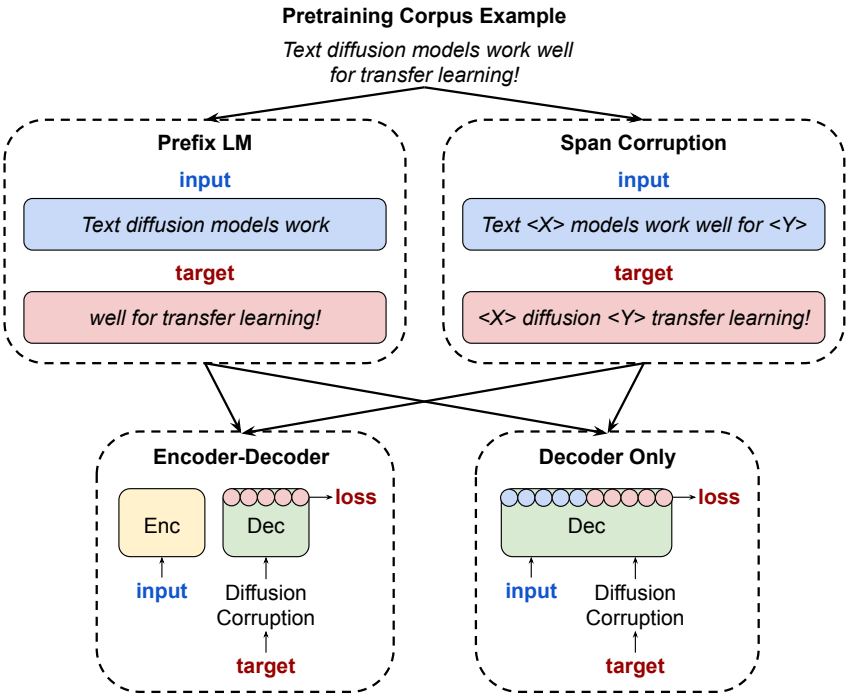

Figure 1: Pretraining objectives and model architectures. The <X> and <Y> symbols are unique sentinel tokens denoting masked spans. Note, the "masking noise" applied to produce the span corruption input/target is independent from the "diffusion noise" which randomly corrupts a subset of target tokens. Loss is only computed over target tokens. In the decoder-only setting, input tokens are frozen when computing the unrolled logits input (l_2).

Architecture	Pretraining Objective	WMT14 En-Fr (BLEU)	SQuAD (F1)	MBPP (Pass@80 %)
Encoder-Decoder	Prefix LM	27.6	75.8	0.0
Decoder-only	Prefix LM	**29.8**	77.4	**12.2**
Encoder-Decoder	Span Corruption	28.7	78.2	0.0
Decoder-only	Span Corruption	29.1	**80.6**	11.1

Table 1: Diffusion model performance on three tasks across model architecture and pretraining objective. The Decoder-only architecture outperforms Encoder-Decoder across all three tasks, despite using fewer parameters.

fine-tune each model separately on WMT14 En-Fr, SQuAD, and MBPP (producing 12 fine-tuned models total) and evaluate across all tasks. We use a fine-tuning batch size of 128 and a constant learning rate of 0.001 across all tasks. We fine-tune 500K steps for WMT14 En-Fr and 250K steps for SQuAD, with checkpoints taken every 1,000 steps. For MBPP due to smaller dataset size, we fine-tune for 5,000 steps with checkpoints taken every 50 steps. In all cases, we terminate fine-tuning if clear evidence of over-fitting is observed. We reuse the 256K token SentencePiece vocabulary from PaLM (Chowdhery et al., 2022). Our decoder-only models have roughly 280M parameters (including embedding parameters), while our encoder-decoder models have roughly 590M parameters.

The results in Table 1 show that our decoder-only models perform the best across all three tasks, despite their lower parameter count. This advantage is especially clear on code synthesis (MBPP), where the encoder-decoder models fail to solve any problem in the test set, even on the permissive "Pass@80" metric that samples the model 80 times and is scored as correct if *any* of these candidates passes. In line with Tay et al. (2023), we suspect that pretraining the model to generate longer contiguous spans is a better-matched objective for downstream tasks like MBPP requiring long coherent generation.

Our findings on pretraining objective are less conclusive, with Prefix LM performing the best on WMT and MBPP, while Span Corruption does best on SQuAD. With this in mind, we select "decoder-only + prefix LM" for our subsequent experiments, as this setup is increasingly standard for LLM training, and does relatively well (best or second-best)

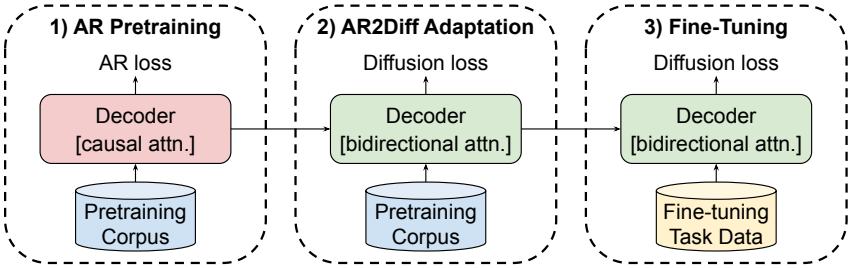

Figure 2: Illustration of our AR2Diff method. 1) Pretrain an AR decoder with causal attention. 2) Continue pretraining as a diffusion model with bidirectional attention. 3) Fine-tune as a diffusion model on the end task.

across all our tasks.

4.3 Transfer learning baselines

We now turn to testing various transfer learning strategies across model scales. As our core baselines, we pretrain both AR and diffusion models at Base (280M), Large (270M), and XL (1.7B) sizes. These all use a decoder-only architecture and prefix LM objective, and train on the same pretraining mixture from the previous section (80% multilingual web pages and 20% Python code). As before, we pretrain for 1M steps, with batch size 128 and sequence length 1024. Note, our diffusion models use bidirectional attention to allow modifying all sequence positions in parallel, but are otherwise architecturally identical to their AR counterparts.

For the AR baselines, at inference time, we use greedy decoding for SQuAD, following T5, and use temperature sampling for MBPP, following Austin et al. (2021). For WMT, we use greedy decoding as opposed to the more commonly used beam search for a fairer comparison, as we did not investigate the use of beam search for diffusion models; see Reid et al. (2023) for work in this direction.

We then fine-tune each of these models separately for each of our three tasks. Results are shown in Table 2, and discussed in section §4.5.

4.4 AR2Diff: Adapting from AR to diffusion

Beyond pure AR and pure diffusion training, we explore "AR2Diff" methods for adapting a pretrained AR model into a diffusion model later in training. First, we experiment with simply fine-tuning an AR checkpoint directly using our diffusion training procedure—enabling bidirectional attention, and using the SUNDAE diffusion training loss. We refer to this method as AR2Diff_0, and use our baseline AR model checkpoint as the starting point for fine-tuning.

We also experiment with pretraining the model for additional steps as a diffusion model *before* fine-tuning, as illustrated in Figure 2. We start with our pretrained AR checkpoint, continue pretraining for an additional N steps using diffusion training, and then fine-tune (still with diffusion) on each evaluation task separately. We refer to this method as AR2Diff_N.

4.5 Core results

Results comparing AR2Diff to our autoregressive and diffusion baselines across model sizes are shown in Table 2.

On WMT14 En-Fr, the AR baseline performs the best across model sizes.[6] Our observed gap between diffusion and AR is larger than that of Savinov et al. (2022), where SUNDAE text diffusion comes with 1 BLEU point of an AR baseline. The difference may be due to our (i) using a transfer learning setting where we pretrain before fine-tuning, (ii) not using SUNDAE's length prediction module, (iii) sampling fewer candidates at inference time (8 vs. 16).

Interestingly, while at Base size AR2Diff provides no advantage on WMT, at Large and XL sizes we see AR2Diff delivers a significant gain over the pure diffusion baseline, and this gain increases with the length of adaptation. This suggests that AR2Diff may be valuable not just as a resource-saving method (leveraging AR checkpoints to avoid pretraining diffusion models from scratch), but also as a means of achieving stronger diffusion models through mixed-objective training.

On SQuAD question answering, our diffusion baseline outperforms the AR baseline at Base and Large sizes (Base: $68.1 \rightarrow 77.4$, Large:

[6]We note our Base AR baseline underperforms (32.27 vs. 37.5) a similar baseline from Raffel et al. (2020), a Base size decoder-only model trained with the same prefix LM objective. This could stem from differences in pretraining data, model architecture, fine-tuning procedure, and/or inference settings (e.g., our use of greedy decoding).

Method	Size	WMT14 En-Fr (BLEU)	SQuAD (F1)	MBPP (Pass@80 %)
Autoregressive	Base	**33.27**	68.11	5.5
Diffusion	Base	29.83	**77.41**	**12.2**
AR2Diff$_0$	Base	29.62	64.77	1.1
AR2Diff$_{10,000}$	Base	29.41	68.12	4.4
AR2Diff$_{100,000}$	Base	29.92	71.87	7.7
Autoregressive	Large	**34.92**	78.43	**15.5**
Diffusion	Large	29.36	80.56	12.2
AR2Diff$_0$	Large	31.14	77.82	3.3
AR2Diff$_{10,000}$	Large	31.97	79.62	8.8
AR2Diff$_{100,000}$	Large	32.20	**80.71**	10.0
Autoregressive	XL	**35.48**	**84.08**	15.5
Diffusion	XL	29.30	82.78	**18.8**
AR2Diff$_0$	XL	32.36	80.95	6.6
AR2Diff$_{10,000}$	XL	32.39	80.71	11.1
AR2Diff$_{100,000}$	XL	32.55	83.54	15.5

Table 2: Performance of various models across three tasks and three sizes, comparing: (i) an AR baseline, (ii) a diffusion baseline, and (iii) AR2Diff models that adapt the pretrained AR baseline via diffusion training for N steps before fine-tuning using diffusion, with $N \in \{0, 10\text{K}, 100\text{K}\}$.

$78.4 \rightarrow 80.6$), but underperforms at XL size ($84.1 \rightarrow 82.8$).[7] While adapting to diffusion only during fine-tuning (AR2Diff$_0$) is ineffective, adapting for N steps before fine-tuning (AR2Diff$_N$) outperforms the AR baseline at most sizes, and improves monotonically with N.

On MBPP code synthesis, diffusion outperforms the AR baseline for two out of three model sizes, including the largest XL size ($15.5 \rightarrow 18.8$). As on other tasks, AR2Diff tends to improve with longer adaptation before fine-tuning.

4.6 Ablations

Our results so far have performed diffusion inference by running 10 steps ("`num_steps`") of denoising over 8 randomly sampled decoder inputs per example ("`num_samples`"). Note, only the output with the highest model score is used for evaluation. Table 3 shows the results of varying `num_steps` $\in \{5, 10, 20\}$ and `num_samples` $\in \{4, 8, 16\}$. On the MBPP code synthesis task, we find that increasing step and samples boosts performance, in line with Savinov et al. (2022). Increasing denoising steps is particularly helpful ($5.5 \rightarrow 16.7$), but at the cost of slower inference. On SQuAD the effect of these parameters is more marginal. More generally, we suspect that additional steps and samples may be helpful on long-form text generation tasks like MBPP that are relatively underspecified (e.g., admitting many correct answers in different styles).

[7] As on WMT, these scores are below the results reported by Raffel et al. (2020) using a similar baseline (85.4). See footnote 6.

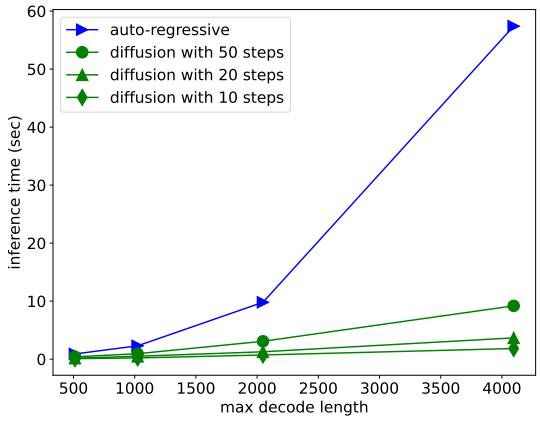

Figure 3: By varying the decoding sequence length, we measure inference time of autoregressive decoding vs. diffusion decoding

By comparison, SQuAD targets are typically short, and are constrained to be spans from the input.

4.7 Inference speed analysis

Diffusion language models have the potential to reduce inference serving costs of long text generation, compared with AR models. Here we show some preliminary results on the inference speed quantitatively. We decode sequences of equal length with AR and diffusion models, and measure corresponding wall-clock times. For diffusion models, we use 10 diffusion steps as our base case, matching our primary evaluation setup for the WMT, SQuAD and MBPP tasks.

We observe an increasing advantage of using diffusion for inference speedup when the generation

Method	steps	samples	SQuAD (F1)	MBPP (Pass@80 %)
Autoregressive	-	-	68.11	5.5
Diffusion	5	8	77.41	5.5
Diffusion	10	8	77.41	12.2
Diffusion	20	8	**77.72**	**16.7**
Diffusion	10	4	**77.51**	11.1
Diffusion	10	8	77.41	12.2
Diffusion	10	16	77.13	**13.3**

Table 3: Ablations on diffusion inference hyperparameters `num_steps` and `num_samples`. Increasing steps and samples leads to clear gains on MBPP, which requires long-form code synthesis, while the effects on SQuAD extractive QA are marginal.

is long. Figure 3 shows as the decoding sequence length increases from 500 tokens (e.g., MBPP task) to 4,000 tokens, the speedup gained by diffusion (using 10 steps) increases from $10\times$ to $30\times$.

Note that a single AR decoding step (14 ms per token generated) is still much faster than a single diffusion step (179 ms per denoising step) in our implementation. This is likely due to the diffusion model's lacking the key-value caching widely used to optimize AR inference. Whether caching or other efficiency optimizations can further extend the speed gains of diffusion is an interesting question for future research.

Acknowledgments

We are grateful to Jiaxin Shi for helpful comments on an earlier draft.

References

Rohan Anil, Andrew M. Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, Eric Chu, Jonathan H. Clark, Laurent El Shafey, Yanping Huang, Kathy Meier-Hellstern, Gaurav Mishra, Erica Moreira, Mark Omernick, Kevin Robinson, Sebastian Ruder, Yi Tay, Kefan Xiao, Yuanzhong Xu, Yujing Zhang, Gustavo Hernandez Abrego, Junwhan Ahn, Jacob Austin, Paul Barham, Jan Botha, James Bradbury, Siddhartha Brahma, Kevin Brooks, Michele Catasta, Yong Cheng, Colin Cherry, Christopher A. Choquette-Choo, Aakanksha Chowdhery, Clément Crepy, Shachi Dave, Mostafa Dehghani, Sunipa Dev, Jacob Devlin, Mark Díaz, Nan Du, Ethan Dyer, Vlad Feinberg, Fangxiaoyu Feng, Vlad Fienber, Markus Freitag, Xavier Garcia, Sebastian Gehrmann, Lucas Gonzalez, Guy Gur-Ari, Steven Hand, Hadi Hashemi, Le Hou, Joshua Howland, Andrea Hu, Jeffrey Hui, Jeremy Hurwitz, Michael Isard, Abe Ittycheriah, Matthew Jagielski, Wenhao Jia, Kathleen Kenealy, Maxim Krikun, Sneha Kudugunta, Chang Lan, Katherine Lee, Benjamin Lee, Eric Li, Music Li, Wei Li, YaGuang Li, Jian Li, Hyeontaek Lim, Hanzhao Lin, Zhongtao Liu, Frederick Liu, Marcello Maggioni, Aroma Mahendru, Joshua Maynez, Vedant Misra, Maysam Moussalem, Zachary Nado, John Nham, Eric Ni, Andrew Nystrom, Alicia Parrish, Marie Pellat, Martin Polacek, Alex Polozov, Reiner Pope, Siyuan Qiao, Emily Reif, Bryan Richter, Parker Riley, Alex Castro Ros, Aurko Roy, Brennan Saeta, Rajkumar Samuel, Renee Shelby, Ambrose Slone, Daniel Smilkov, David R. So, Daniel Sohn, Simon Tokumine, Dasha Valter, Vijay Vasudevan, Kiran Vodrahalli, Xuezhi Wang, Pidong Wang, Zirui Wang, Tao Wang, John Wieting, Yuhuai Wu, Kelvin Xu, Yunhan Xu, Linting Xue, Pengcheng Yin, Jiahui Yu, Qiao Zhang, Steven Zheng, Ce Zheng, Weikang Zhou, Denny Zhou, Slav Petrov, and Yonghui Wu. 2023. Palm 2 technical report.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models.

Ondřej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, Radu Soricut, Lucia Specia, and Aleš Tamchyna. 2014. Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the Ninth Workshop on Statistical Machine Translation*, pages 12–58, Baltimore, Maryland, USA. Association for Computational Linguistics.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners.

Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. 2023.

Analog bits: Generating discrete data using diffusion models with self-conditioning. In *The Eleventh International Conference on Learning Representations*.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways.

Sander Dieleman, Laurent Sartran, Arman Roshannai, Nikolay Savinov, Yaroslav Ganin, Pierre H. Richemond, Arnaud Doucet, Robin Strudel, Chris Dyer, Conor Durkan, Curtis Hawthorne, Rémi Leblond, Will Grathwohl, and Jonas Adler. 2022. Continuous diffusion for categorical data.

Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. 2019. Mask-predict: Parallel decoding of conditional masked language models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 6112–6121, Hong Kong, China. Association for Computational Linguistics.

Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. 2023. Diffuseq: Sequence to sequence text generation with diffusion models.

Jiatao Gu, James Bradbury, Caiming Xiong, Victor O.K. Li, and Richard Socher. 2018. Non-autoregressive neural machine translation. In *International Conference on Learning Representations*.

Jiatao Gu and Xiang Kong. 2021. Fully non-autoregressive neural machine translation: Tricks of the trade. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 120–133, Online. Association for Computational Linguistics.

Yoon Kim and Alexander M. Rush. 2016. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1317–1327, Austin, Texas. Association for Computational Linguistics.

Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. The stack: 3 tb of permissively licensed source code.

Jason Lee, Elman Mansimov, and Kyunghyun Cho. 2018. Deterministic non-autoregressive neural sequence modeling by iterative refinement. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1173–1182, Brussels, Belgium. Association for Computational Linguistics.

Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori Hashimoto. 2022. Diffusion-LM improves controllable text generation. In *Advances in Neural Information Processing Systems*.

Zhenghao Lin, Yeyun Gong, Yelong Shen, Tong Wu, Zhihao Fan, Chen Lin, Nan Duan, and Weizhu Chen. 2023. Text generation with diffusion language models: A pre-training approach with continuous paragraph denoise.

OpenAI. 2023. Gpt-4 technical report.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical text-conditional image generation with clip latents.

Machel Reid, Vincent Josua Hellendoorn, and Graham Neubig. 2023. DiffusER: Diffusion via edit-based reconstruction. In *The Eleventh International Conference on Learning Representations*.

Adam Roberts, Hyung Won Chung, Anselm Levskaya, Gaurav Mishra, James Bradbury, Daniel Andor, Sharan Narang, Brian Lester, Colin Gaffney, Afroz Mohiuddin, Curtis Hawthorne, Aitor Lewkowycz, Alex Salcianu, Marc van Zee, Jacob Austin, Sebastian Goodman, Livio Baldini Soares, Haitang Hu, Sasha Tsvyashchenko, Aakanksha Chowdhery, Jasmijn Bastings, Jannis Bulian, Xavier Garcia, Jianmo Ni, Andrew Chen, Kathleen Kenealy, Jonathan H. Clark, Stephan Lee, Dan Garrette, James Lee-Thorp, Colin Raffel, Noam Shazeer, Marvin Ritter, Maarten Bosma, Alexandre Passos, Jeremy Maitin-Shepard, Noah Fiedel, Mark Omernick, Brennan Saeta, Ryan Sepassi, Alexander Spiridonov,

Joshua Newlan, and Andrea Gesmundo. 2022. Scaling up models and data with t5x and seqio. *arXiv preprint arXiv:2203.17189*.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2021. High-resolution image synthesis with latent diffusion models. *CoRR*, abs/2112.10752.

Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Raphael Gontijo-Lopes, Burcu Karagol Ayan, Tim Salimans, Jonathan Ho, David J. Fleet, and Mohammad Norouzi. 2022. Photorealistic text-to-image diffusion models with deep language understanding. In *Advances in Neural Information Processing Systems*.

Chitwan Saharia, William Chan, Saurabh Saxena, and Mohammad Norouzi. 2020. Non-autoregressive machine translation with latent alignments. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1098–1108, Online. Association for Computational Linguistics.

Nikolay Savinov, Junyoung Chung, Mikolaj Binkowski, Erich Elsen, and Aaron van den Oord. 2022. Step-unrolled denoising autoencoders for text generation. In *International Conference on Learning Representations*.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. 2015. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 2256–2265, Lille, France. PMLR.

Mitchell Stern, William Chan, Jamie Kiros, and Jakob Uszkoreit. 2019. Insertion transformer: Flexible sequence generation via insertion operations. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5976–5985. PMLR.

Robin Strudel, Corentin Tallec, Florent Altché, Yilun Du, Yaroslav Ganin, Arthur Mensch, Will Grathwohl, Nikolay Savinov, Sander Dieleman, Laurent Sifre, and Rémi Leblond. 2022. Self-conditioned embedding diffusion for text generation.

Yi Tay, Mostafa Dehghani, Vinh Q. Tran, Xavier Garcia, Jason Wei, Xuezhi Wang, Hyung Won Chung, Dara Bahri, Tal Schuster, Steven Zheng, Denny Zhou, Neil Houlsby, and Donald Metzler. 2023. UL2: Unifying language learning paradigms. In *The Eleventh International Conference on Learning Representations*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

Tong Wu, Zhihao Fan, Xiao Liu, Yeyun Gong, Yelong Shen, Jian Jiao, Hai-Tao Zheng, Juntao Li, Zhongyu Wei, Jian Guo, Nan Duan, and Weizhu Chen. 2023. Ar-diffusion: Auto-regressive diffusion model for text generation.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498, Online. Association for Computational Linguistics.

Jiasheng Ye, Zaixiang Zheng, Yu Bao, Lihua Qian, and Quanquan Gu. 2023. Diffusion language models can perform many tasks with scaling and instruction-finetuning.

Hongyi Yuan, Zheng Yuan, Chuanqi Tan, Fei Huang, and Songfang Huang. 2023. Seqdiffuseq: Text diffusion with encoder-decoder transformers.

Lin Zheng, Jianbo Yuan, Lei Yu, and Lingpeng Kong. 2023. A reparameterized discrete diffusion model for text generation.